Stage 4 实验报告

于越洋

1 实验内容

Step 9

词法分析

考虑到函数参数列表中需要用逗号隔开不同的参数，因此我在 `frontend/lexer/lex.py` 中添加了 `t_Comma=','` 。
我也在 `frontend/parser/ply_parser.py` 中定义了新的规则：

- `p_params`： 表示函数定义的参数列表，包含多个参数 `parameter`；
- `p_params_empty`： 表示空参数列表；
- `p_parameter`： 表示一个函数参数，为 `type Identifier` 的形式；
- `p_function_call`： 表示函数调用语句，包含函数标识符 `Identifier` 和实参列表 `arguments`；
- `p_arguments`： 表示函数调用时的实参列表，包含多个表达式 `expression`；
- `p_arguments_empty`： 表示空实参列表。

修改以下已有规则：

- `p_program`： 现在一个程序 `program` 可以由多个函数 `function` 构成；
- `p_function_def`： 现在函数定义包含参数列表 `params` 。

语法分析

在 `frontend/ast/tree.py` 中，我添加了新的 AST 节点类型：

- `ParameterList`： 用于表示函数参数列表；
- `Parameter`： 用于表示函数参数，包含类型 `var_t` 和变量标识符 `ident`；
- `Call`： 用于表示函数调用，包含函数标识符 `ident` 和实参列表 `arguments`；
- `ArgumentList`： 用于表示函数实参列表。

修改了已有的节点类型：

- `Program`： 增加了成员函数 `functions_list`，用于以列表形式返回所有函数节点，便于对同名函数多次声明和重定义进行处理；
- `Function`： 增加了参数列表 `params` 。

语义分析

在 `frontend/scope/scope.py` 中，我为 `ScopeKind` 类添加了 `FUNC_DECL` 成员，表示函数声明作用域。

在 `frontend/scope/scopestack.py` 中，我在 `ScopeStack` 类中添加了以下成员函数：

- `lookupInFuncScope`： 用于在函数内部查询已声明的符号，一方面，查询本层局部作用域内是否声明了同名符号；另一方面，如果本层局部作用域的上一层是函数声明作用域，则还会查询上一层作用域内是否声明了同名符号，以便处理函数参数作用域。

在 `frontend/typecheck/namer.py` 中，我在 `Namer` 类中添加了以下成员函数：

- `visitParameter`： 用于对函数参数进行语义分析，先检查函数声明作用域中是否有同名变量符号，若有则报声明冲突错误，否则声明新的变量。
- `visitCall`： 用于对函数调用语句进行语义分析，先检查调用的函数是否已被声明，再检查传入实参的数量是否与函数接收参数的数量相等，若未通过以上检查将会报编译错误，否则设定调用语句的结果为函数返回值，并逐个对实参列表 `arguments` 中的每个参数表达式进行语义分析。

我还修改了 `Namer` 类的以下成员函数：

- `visitProgram`： 对程序中的所有函数进行语义分析时，使用 `functions_list()` 代替原来的 `functions().values()` 作为待分析函数列表。
- `visitFunction`： 若目标函数符号已被定义，则检查函数声明是否与已有定义不一致，若是则报声明冲突错误，否则认为是函数重复声明，通过检查；若目标函数符号未被定义，则定义新的函数符号，将参数列表 `params` 中的每个参数添加进函数声明作用域 `Scope(ScopeKind.FUNC_DECL)`，并在函数符号 `funcSymbol` 中记录参数列表中所有参数的类型，最后对函数体进行语义分析。
- `visitDeclaration`： 在检查变量符号是否已经被声明过时，使用 `lookupInFuncScope` 函数检查同作用域和上层函数声明作用域（若上层是函数声明作用域）内的声明。

中间代码生成

在 `utils/tac/tacop.py` 中，我为 `InstrKind` 添加了 `PARAM` 类型，表示函数参数。

在 `utils/tac/tacfunc.py` 的 `TACFunc` 类中，我修改了 `printTo` 函数，跳过了对函数参数 `Parameter` 的打印。

在 `utils/tac/tacinstr.py` 中，我在 `TACInstr` 基类中添加了 `isParameter` 函数判断是否为函数参数语句。

我还添加了如下 TAC 语句类型：

- `Call`：表示函数调用语句，包含函数标签 `funcLabel`，参数临时变量列表 `args`，返回值临时变量 `ret`，并将函数调用的 TAC 语句设定为 `ret = CALL funcLabel(arg1, arg2, ...)` 的形式；
- `Parameter`：表示函数参数，包含一个参数临时变量 `arg`，无需在 TAC 中打印。

在 `frontend/tacgen/tacgen.py` 中，我为 `TACGen` 类添加了以下成员函数：

- `visitFunction`：先逐个接受参数列表 `params` 中的每个参数，再接受函数体；
- `visitParameter`：为参数符号分配临时变量，生成 `Parameter` 语句；
- `visitCall`：先逐个接受实参列表 `arguments` 中每个形参表达式，再接受 `Call` 语句；
- `visitArgument`：直接接受实参表达式；

我也修改了 `TACGen` 类中的以下成员函数：

- `transform`：将 `emitter` 初始化时传入的参数数量设定为 `len(astFunc.params)`，然后接受 `astFunc`，而不是接受 `astFunc.body`。

我还在 `TACFuncEmitter` 类中添加了以下成员函数：

- `emitCall`：为函数返回值分配临时变量，在指令序列中添加 `Call` 语句；
- `emitParameter`：在指令序列中添加 `Parameter` 语句。

目标代码生成

我在 `utils/riscv.py` 中为 `Riscv` 类添加了以下成员类：

- `Call`：表示函数调用语句，打印为 `call funcLabel`；
- `Parameter`：表示函数参数语句，打印为空语句（实际不会被打印）；

我还在 `backend/riscv/riscvasmemitter.py` 中的 `RiscvInstrSelector` 类中添加了以下成员函数：

- `visitCall`：在 RISC-V 代码序列中添加函数调用语句；
- `visitParameter`：在RISC-V 代码序列中添加函数参数语句。

并修改了 `RiscvSubroutineEmitter` 类中的以下成员函数：

- `emitFunc`：添加了对 `ra` 寄存器的保存和恢复。

最后，我修改了在 `backend/reg/bruteregalloc.py` 中的 `BruteRegAlloc` 类的以下成员函数：

- `localAlloc`：增加了对于函数调用和函数参数的处理。
  - 函数调用：首先逐个清空参数寄存器，放入实参；再保存传参过程未使用的 caller-saved 寄存器；再将超过 8 个的实参从右向左逐个压入栈中传递，全部压栈完成后统一

修改栈指针；最后进行函数调用，保存函数返回值，恢复由传参修改的栈指针。
- 函数参数：对于通过寄存器传递的参数，直接将临时变量和寄存器进行绑定；对于通过栈传递的参数，将临时变量绑定到参数在栈中的位置。

2 思考题

Step 9

Question 1

> 你更倾向采纳哪一种中间表示中的函数调用指令的设计（一整条函数调用 vs 传参和调用分离）？写一些你认为两种设计方案各自的优劣之处。
> 具体而言，某个“一整条函数调用”的中间表示大致如下：

```
_T3 = CALL foo(_T2, _T1, _T0)
```

> 对应的“传参和调用分离”的中间表示类似于：

```
PARAM _T2
PARAM _T1
PARAM _T0
_T3 = CALL foo
```

Answer：我更倾向于“一整条函数调用”，在我的实现中也是这样做的。

“一整条函数调用”设计方案
优点：参数（Def）和返回值（Use）在一条指令中定义，方便构建 Def-Use 链；天然适配静态单赋值形式，利于数据流分析；调用过程作为一个整体，指令调度时不会错误地将其他指令插入参数准备和跳转之间。
缺点：与机器调用序列差别较大；操作数数量不固定，数据结构设计相对复杂。

“传参和调用分离”设计方案
优点：更贴近生成机器调用序列；指令定长，便于解析和存储。
缺点：`PARAM` 和 `CALL` 逻辑强相关但物理分离，增加活跃变量分析和寄存器分配的难度；指令调度器必须小心维护顺序，防止参数被覆盖。

Question 2

> 为何 RISC-V 标准调用约定中要引入 callee-saved 和 caller-saved 两类寄存器，而不是要求所有寄存器完全由 caller/callee 中的一方保存？为何保存返回地址的 ra 寄存器是 caller-saved 寄存器？

Answer：引入 callee-saved 和 caller-saved 两类寄存器是为了减少不必要的保存/恢复开销。如果全部为 caller-saved 寄存器，每次调用，调用者不知道被调用者需要使用哪些寄存器，只能保存大量活跃寄存器，开销大；如果全部为 callee-saved 寄存器，每个函数入口/出口都要保存一堆可能之后不会再使用的寄存器，代价过高；如果混合使用，将跨函数调用的长生命周期变量放入 callee-saved 寄存器，不跨函数调用的短生命周期变量放入 caller-saved 寄存器，节省保存/恢复开销。

保存返回地址的 `ra` 寄存器是 caller-saved 寄存器的原因为：RISC-V 的跳转指令 `jal/jalr` 会自动覆盖 `ra`，对于非叶子函数，调用别人时当前的 `ra` 必然会被覆盖，因此需要调用者在覆盖前手动保存它；当 `ra` 是 caller-saved 寄存器时，叶子函数不需要保存 `ra`（因为不需要调用其他函数），更加高效。